Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms
Chapter 4. Greedy Algorithms
Session 1

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

Chapter Outline

Chapter 4. Greedy Algorithms

7 hours

4.1 Basic concepts
4.2 Fractional knapsack problem
4.3 Job sequencing with deadlines
4.4 Analysis of minimum spanning trees related algorithms
4.5 Analysis of single source shortest path algorithm

Optimization Problems and Optimal Solutions

→ In mathematics and engineering, an optimization problem is the problem of finding the best solution from all feasible solutions.

→ Optimization problems can be divided into two categories whether the variables are continuous or discrete.

→ An optimization problem with discrete variables is known as discrete optimization.

→ An optimal solution is a feasible solution where the objective function reaches its maximum(or minimum) value- for example, the most profit or the least cost.

→ A globally optimal solution is one where there are no other feasible solutions with better objective function value.

→ A locally optimal solution is one where there are no other feasible solutions in the vicinity with the better objective function values.

Introduction of Greedy Algorithms

→ Greedy is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit.

→ So the problems where choosing locally optimal also leads to global solution are the best fit for Greedy.

→ The algorithm never reverses the earlier decision even if the choice is wrong. It works in a top-down approach.

→ This algorithm may not produce the best result for all the problems. It's because it always goes for the local best choice to produce the global best result.

→ A greedy algorithm works is a problem exhibits the following properties:

- ❑ **Greedy Choice Property:** A globally optimal solution can be reached by creating a locally optimal solution. In other words, an optimal solution can be obtained by creating "greedy" choices.
- ❑ **Optimal substructure:** Optimal solutions contain optimal sub-solutions. In other words, answers to sub-problems of an optimal solution are optimal.

Areas of Applications

→ Greedy approach is used to solve many problems, such as:

- ❑ Finding the shortest path between two vertices using Dijkstra's algorithm.
- ❑ Finding the minimum spanning tree in a graph using Prim's/Kruskal's algorithm etc.

Elements of Greedy Strategy

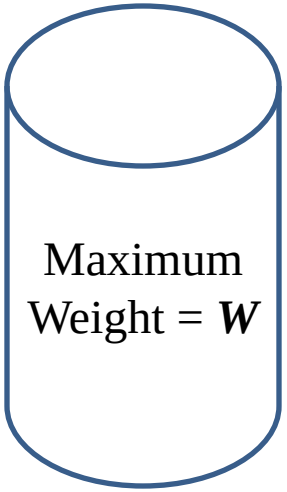
→ Greedy algorithms have the following five components:

- **A candidate set:** A solution is created from this set.
- **A selection function:** Used to choose the best candidate to be added to the solution.
- **A feasibility function:** Used to determine whether a candidate can be used to contribute to the solution.
- **An objective function:** Used to assign a value to a solution or a partial solution.
- **A solution function:** Used to indicate whether a complete solution has been reached.

Fractional Knapsack

→ **Statement:**

A thief has a bag or knapsack that can contain maximum weight ***W*** of his loot. There are ***n*** items and the weight of ***i^{th}*** item is w_i and its worth is v_i. Any amount of item can be put into the bag. i.e. x_i fraction of item can be collected ,where $0 \leq x_i \leq 1$. Here, the objective is to collect the items that maximize the total profit earned. Here, we arrange the items by the ratio v_i/w_i.



Knapsack or a bag

$$\max \sum_{i=1}^{n} x_i v_i$$

Subject to

$$\sum_{i=1}^{n} w_i x_i \leq W$$

$$0 \leq x_i \leq 1$$

Object	1	2	3	4	5	6	7
Worth	v_1	v_2	v_3	v_4	v_5	v_6	v_7
Weight	w_1	w_2	w_3	w_4	w_5	w_6	w_7
Profit	v_1/w_1	v_2/w_2	v_3/w_3	v_4/w_4	v_5/w_5	v_6/w_6	v_7/w_7

Example:

Object	1	2	3	4	5	6	7
Worth	10	5	15	7	6	18	3
Weight	2	3	5	7	1	4	1
Profit	5	1.66	3	1	6	4.5	3

→ **Algorithm:**

```
Algorithm FractionalKnapsack(W,n)
{
    for(i=1;i<=n;i++)
        x[i]=0.0;
    tempW=W;
    for(i=1;i<=n;i++)
    {
    if(w[i]>tempW) then
        break;
    x[i]=1.0;
    tempW-=w[i];
    }
    if(i<=n)
        x[i]=tempW/w[i];
}
```

→ **Analysis:**

We can see that the above algorithm just contain a single loop. i.e. no nested loops. The running time for above algorithm is $O(n)$. However, our requirement is that $v[1\ldots n]$ and $w[1\ldots n]$ are sorted, so we can use sorting methods to sort it in $O(n \log n)$ time such that the complexity of the algorithm above including sorting becomes $O(n \log n)$.

Example

***Question*:**

Consider five items along with their respective weights and values,

I = {I1, I2, I3, I4, I5}

w = {5, 10, 20, 30, 40}

v = {30, 20, 100, 90, 160}

If the knapsack has capacity W = 60, then find the optimal profit earned by using the fractional knapsack algorithm.

***Solution*:**

Initially,

Items	w_i	v_i
I1	5	30
I2	10	20
I3	20	100
I4	30	90
I5	40	160

Calculating v_i/w_i, we get,

Items	w_i	v_i	$P_i = v_i/w_i$
I1	5	30	6.0
I2	10	20	2.0
I3	20	100	5.0
I4	30	90	3.0
I5	40	160	4.0

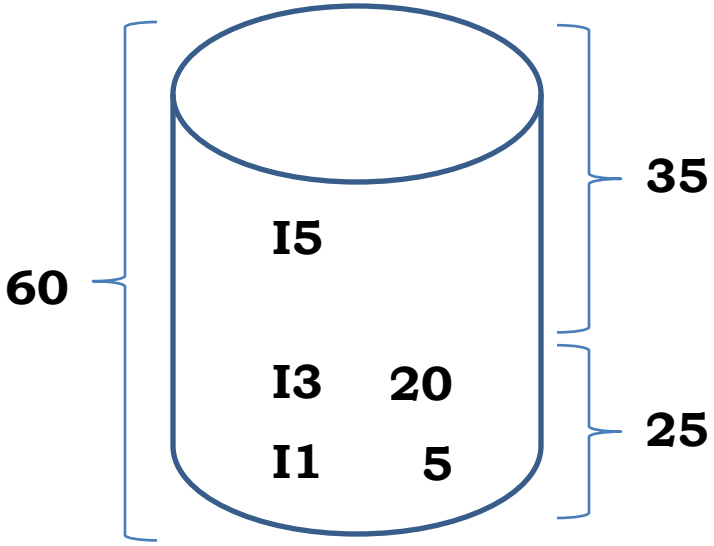
Example

Arranging the items in the decreasing order of P_i, we get,

Items	w_i	v_i	$P_i = v_i/w_i$
I1	5	30	6.0
I3	20	100	5.0
I5	40	160	4.0
I4	30	90	3.0
I2	10	20	2.0

Items	x_i
I1	1
I3	1
I5	0.875
I4	0
I2	0

Now filling the knapsack according to the decreasing value of P_i, we get,



For object I5,

$40\ \text{W} = 160\ \text{Value}$

$$\therefore 35\ \text{W} = \frac{160}{40} \times 35\,\text{value} = 140\,\text{value}$$

$$\text{Proportion of I5} = \frac{35}{40} = 0.875$$

Total value in the knapsack
= 30 + 100 + 140 = 270

Cross check:

$$\sum_{i=1}^{n} x_i v_i = x_1 v_1 + x_2 v_2 + x_3 v_3 + x_4 v_4 + x_5 v_5$$

$$= 1 \times 30 + 0 \times 20 + 1 \times 100 + 0 \times 90 + 0.875 \times 160$$

$$= 270$$

$$\sum_{i=1}^{n} x_i w_i = 1 \times 5 + 0 \times 10 + 1 \times 20 + 0 \times 30 + 0.875 \times 40 = 60$$

$$\sum_{i=1}^{n} x_i w_i \leq W = 60$$

The objective function is maximized and the constraint is satisfied.

Job Sequencing with Deadlines

→ The problem is the number of jobs , their profit and deadlines will be given and we have to find a sequence of jobs, which will be completed within its deadlines and it should yield a maximum profit.

→ Let there are a number of jobs J = {J1, J2, J3, J4, ….., Jn}

→ Deadline of Jobs = {d1,d2,d3,d4,…..,dn}

→ The profit can be earned if the job is completed within their deadlines = {p1,p2,….,pn}

→ Here every job can be completed in unit time(i.e. first job begins at time 0 and finished at time 1, the second job begins at time 1 and finished at time 2 and so on.) and we have a single machine(processor). The main aim of the problem is to find the feasible sequence of jobs that maximize the profit earned.

→ **Features of the algorithm:**

- There are ***n*** jobs to be processed on a machine.
- Each job *i* has a deadline $d_i \geq 0$ and profit $P_i \geq 0$
- P_i is earned *iff* the job is completed by its deadline.
- The job is completed if it is processed on a machine for a unit time.
- Only one machine is available for processing jobs.
- Only one job is processed at a time on the machine.

Example

Example 1: Let us consider a set of given jobs as shown in the following table. We have to find a sequence of jobs, which will be competed within their deadlines and will given maximum profit. Each job is associated with a deadline and profit.

Job	J1	J2	J3	J4	J5
Deadline	2	1	3	2	1
Profit	60	100	20	40	20

Solution:

To solve this problem, the given jobs are sorted according to their profit in descending order.

Job	J2	J1	J4	J3	J5
Deadline	1	2	2	3	1
Profit	100	60	40	20	20

Job	J2	J1	J4	J3	J5
Deadline	1	2	2	3	1
Profit	100	60	40	20	20

Job	**Feasible/Non Feasible**	**Processing Sequence**	**Total Profit**
J2	Feasible	{J2, -, -}	100
J1	Feasible	{J2, J1, -}	100+60=160
J4	Not-Feasible	{J2, J1, -}	160
J3	Feasible	{J2, J1, J3}	160+20=180
J5	Not Feasible	{J2, J1, J3}	180

→ From this set of jobs, first we select J2, as it can be completed within its deadline and contributes maximum profit.

→ Next, J1 is selected as it gives more profit compared to J4.

→ In the next clock, J4 cannot be selected as its deadline is over, hence J3 is selected as it executes within its deadline.

→ The job J5 is discarded as it cannot be executed within its deadline.

→ Thus, the solution is the sequence of jobs {J2, J1, J3}, which are being executed within their deadline and gives maximum profit.

→ Total profit of this sequence is 100 + 60 + 20 = **180**

Algorithm:

Let us consider, a set of *n* jobs which are associated with deadlines and profit is earned, if a job is completed by its deadline. These jobs need to be ordered in such a way that there is a maximum profit. Assume, deadline of ith job J_i is D_i and the profit received from this job is P_i. Hence, the optimal solution of this algorithm is a feasible solution with maximum profit.

```
Algorithm Job_Sequencing_with_Deadline(D,J,n,k)
{
    D(0)=J(0)=0
    k=1
    J(1)=1 //means first job is selected
    for i=2...n do
        r=k
        while D(J(r))>D(i) and D(J(r))!=r do
            r=r-1
        if D(J(r))<= D(i) and D(i)>r then
            for l=k...r+1 by -1 do
                J(l+1)=J(l)
                J(r+1)=i
                k=k+1
}
```

Step 1. Start
Step 2. Input the number of jobs and their respective deadlines and profits.
Step 3. Sort the jobs in descending order of profit (highest profit first).
Step 4. Find the maximum deadline among all jobs.
Step 5. Create a schedule array of size equal to the maximum deadline and initialize all slots as empty.
Step 6. For each job in sorted order:
6.1.Check from its deadline to earlier slots to find an available slot.
6.2.If a free slot is found, place the job in that slot.
6.3.Add the profit of this job to the total profit counter.
Step 7. Output the final job schedule and total maximum profit.
Step 8. End

Analysis:

In this algorithm, we are using two loops, one within another. Hence, the complexity of the algorithm is $O(n^2)$

Bachelor of Engineering (B.E.) in
Computer Engineering (BCT)
Electronics, Communication & Information Engineering (BEI)
Third Year Second Part
Elective II Course

Analysis of Algorithms
Chapter 4. Greedy Algorithms
Session 2

by:

Er. Bikal Adhikari

(*B.E., M.Sc. Engg., M.Sc. ISE*)

Assistant Professor, Dept. of Electronics & Computer Engg.

Pulchowk Campus, IOE, TU

Chapter Outline

Chapter 4. Greedy Algorithms

7 hours

4.1 Basic concepts
4.2 Fractional knapsack problem
4.3 Job sequencing with deadlines
4.4 Analysis of minimum spanning trees related algorithms
4.5 Analysis of single source shortest path algorithm

Greedy Algorithms

→ Greedy is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit. So the problems where choosing locally optimal also leads to global solution are the best fit for Greedy.

→ The algorithm never reverses the earlier decision even if the choice is wrong. It works in a top-down approach.

→ This algorithm may not produce the best result for all the problems. It's because it always goes for the local best choice to produce the global best result.

Shortest Path Algorithm

→ Weighted graph is a graph in which values are assigned to the edges and the length of the path is the sum of weights of the edges in the path.

→ Let w(i,j) denote the weight of edge(i,j).

→ In weighted graph, we often want to find the shortest path(i.e. path having a minimum length) between two given vertices.

→ Algorithm due to E.W. Dijkstra, which efficiently solves this problem also known as Dijkstra's algorithm is discussed next.

Dijkstra's Shortest Path Algorithm

→ This algorithm finds the length of a shortest path from vertex ***a*** to vertex ***z*** in a connected and weighted graph. The weight of edge (***i***,***j***) is ***w***(***i***,***j***)>0 and the label of vertex ***x*** is ***L***(***x***).

→ At termination, ***L***(***z***) is the length of the shortest path from ***a*** to ***z***.

→ Input: A connected weighted graph in which all weights are positive; vertices ***a*** and ***z***.

→ Output: ***L***(***z***), the length of shortest path from ***a*** to ***z***.

```
1.  dijkstra(w, a, z, L){
2.  L(a)=0
3.  for all vertices x ≠ a
4.          L(x)= ∞
5.  T=set of all vertices
6.  // T is the set of vertices whose shortest distance from a has
7.  //not been found
8.  while(z ∈T){
9.          Choose v ∈T with minimum L(v)
10.         T=T-{v}
11.         for each x ∈T adjacent to v
12.                 L(x)=min{L(x),L(v)+w(v,x)}
13. }
14. }
```

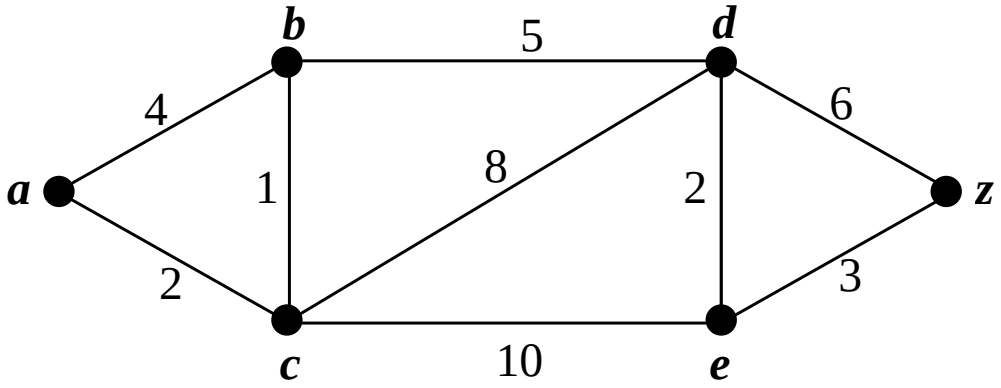
Analysis:

In the above algorithm, the first for loop block takes O(T) time. Initialization of the priority Queue takes O(T) time. The while loop executes for O(T), where for each execution the block inside the loop takes O(T) times. Hence the total running time is $O(T^2)$.

Disadvantages of Dijkstra's Algorithm:

- It does a blind search, so wastes a lot of time while processing.
- It cannot handle negative edges.
- It leads to the acyclic graph and most often cannot obtain the right shortest path.
- We need to keep track of vertices that have been visited.

Example: Use Dijkastra's algorithm to find the length of the shortest path between the vertices a and z in the weighted graph displayed below:



Solution:

Initially,

$L(a) = 0$

For $x \neq a$, $L(x)$ = infinity

$T = \{b,c,d,e,z\}$

First Iteration:

z∈T is true

c ∈T has minimum distance.

T=T-{c} = {b,d,e,z}

L(c)=min{L(c), L(a)+w(a,c)}=min(∞,0+2) = 2

L(b)=min{L(b), L(a)+w(a,b)}=min(∞,0+4) = 4

Second Iteration:

z∈T={b,d,e,z} is true

b ∈T has minimum distance.

T=T-{b} = {d,e,z}

L(b)=min{L(b), L(c)+w(c,b)}=min(4,2+1) = 3

L(d)=min{L(d), L(c)+w(c,d)}=min(∞,2+8) = 10

L(e)=min{L(e),L(c)+w(c,e)}=min(∞,2+10)=12

Third Iteration:

z∈T={d,e,z} is true

d ∈T is the nearest one.

T=T-{d} = {e,z}

L(d)=min{L(d), L(b)+w(b,d)}=min(10,3+5) = 8

Fourth Iteration:

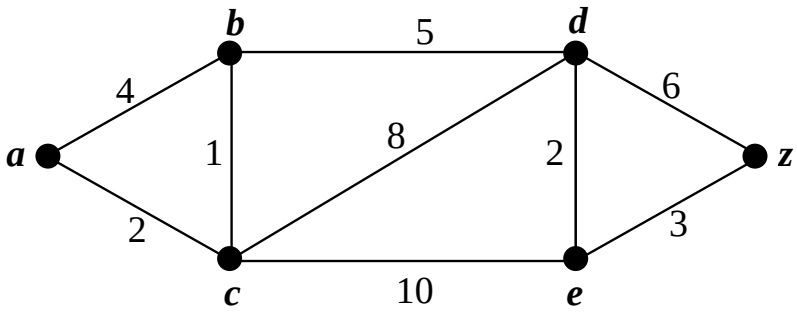
z∈T={e,z} is true

e ∈T is the nearest one.

T=T-{e} = {z}

L(e)=min{L(e), L(d)+w(d,e)}=min(∞,8+2) = 10

L(z)=min{L(z),L(d)+w(d,z)}=min(∞,8+6)=14



Fifth Iteration:

z∈T={z} is true

z ∈T is the nearest one.

T=T-{z} = Φ

L(z)=min{L(z),L(e)+w(e,z)}

=min(14,10+3)=13

Therefore, the length of shortest path(i.e. acbdez) from a to z is 13.

Spanning Tree

→ A ***spanning tree T*** of a graph ***G*** is a tree (or a special sub graph of G) that connects all the ***vertices of a graph*** with the minimum possible ***number of edges.***

→ A spanning tree $T(V', E')$ is a special subgraph of a connected graph $G(V, E)$ such that:

- $V' = V$
- No cycle is formed.
- No. of vertices in spanning tree($|V'|$) = $|V|$
- No. of edges in spanning tree(E') = $|V|$-1

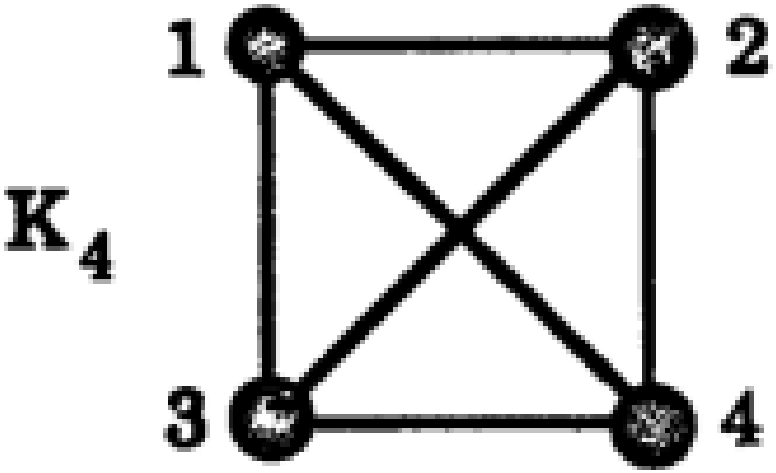
→ A ***complete graph*** **(K_n)** has $|V|^{|V|-2}$ ***spanning trees***.

→ A graph is connected ***iff*** it has a spanning tree.

→ ***Properties of spanning trees:***

- A connected graph G can have more than one spanning tree.
- All possible spanning trees of graph G, have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops).
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is **minimally connected**.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is **maximally acyclic**.

Q. Find all the spanning trees for a complete graph K_4



G=(V,E)

E={(1,2),(2,3),(2,4),(3,4),(1,3),(1,4)}

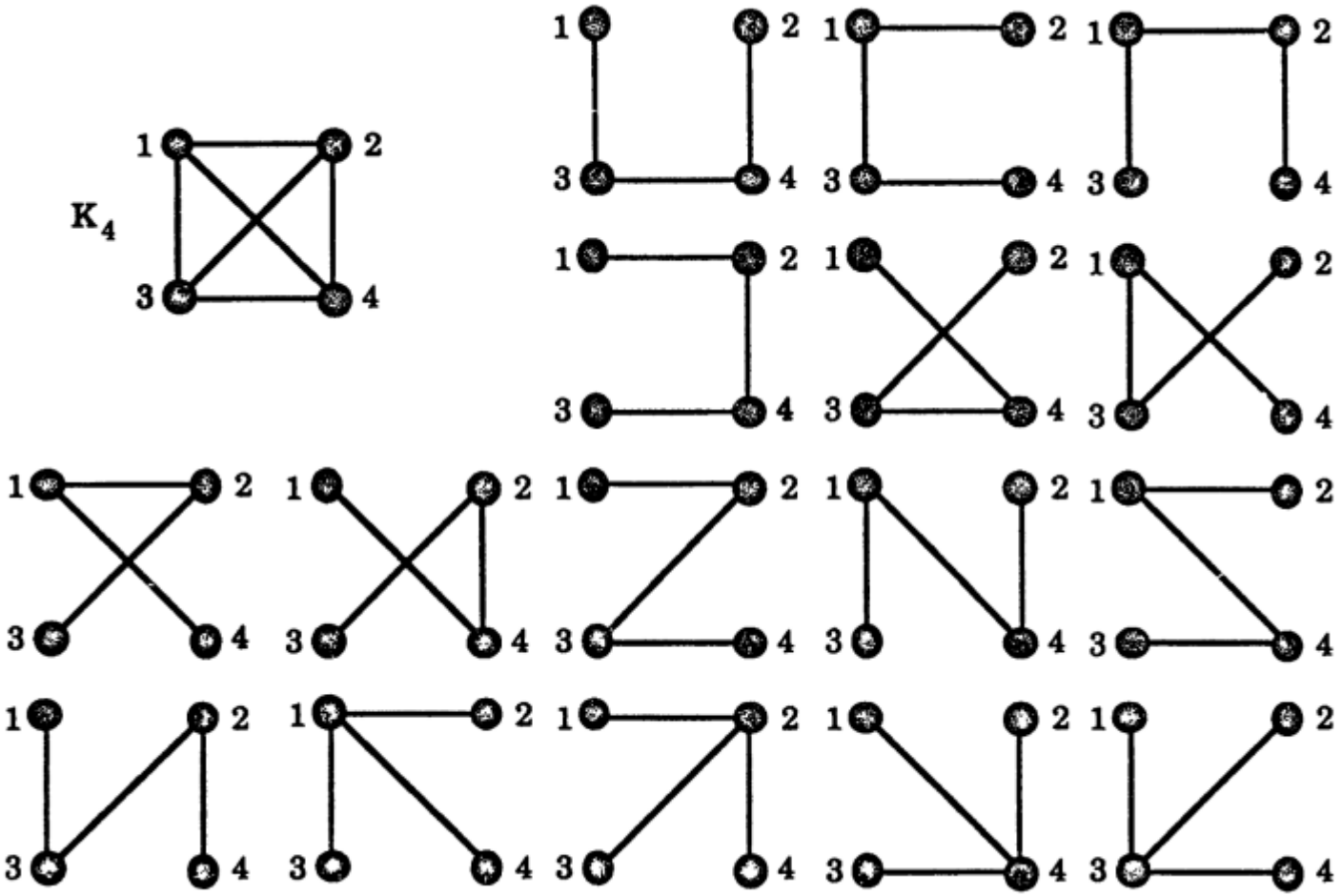
V={1,2,3,4}

|V| = 4

|E| = 6

Solution:

→ The diagram of all possible spanning trees from a graph K_4 are:

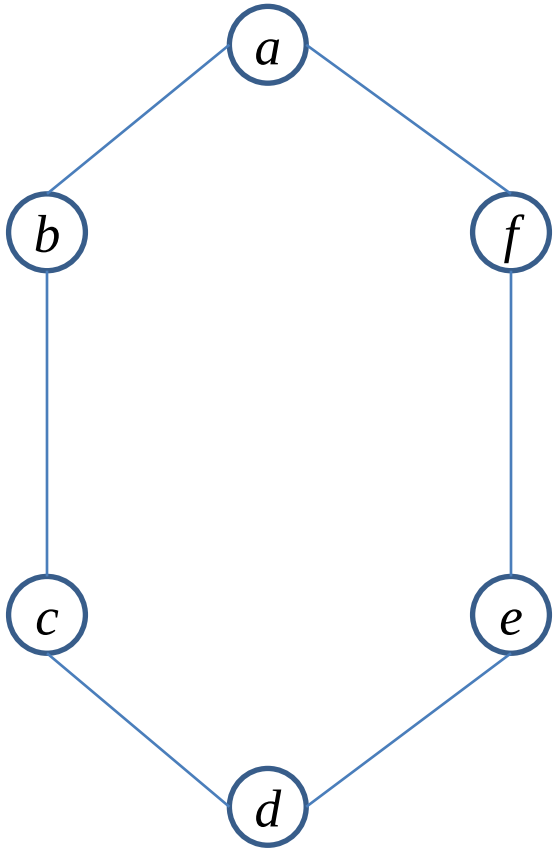


No. of spanning trees = $|V|^{|V|-2} = 4^{4-2}=4^2 = 16$

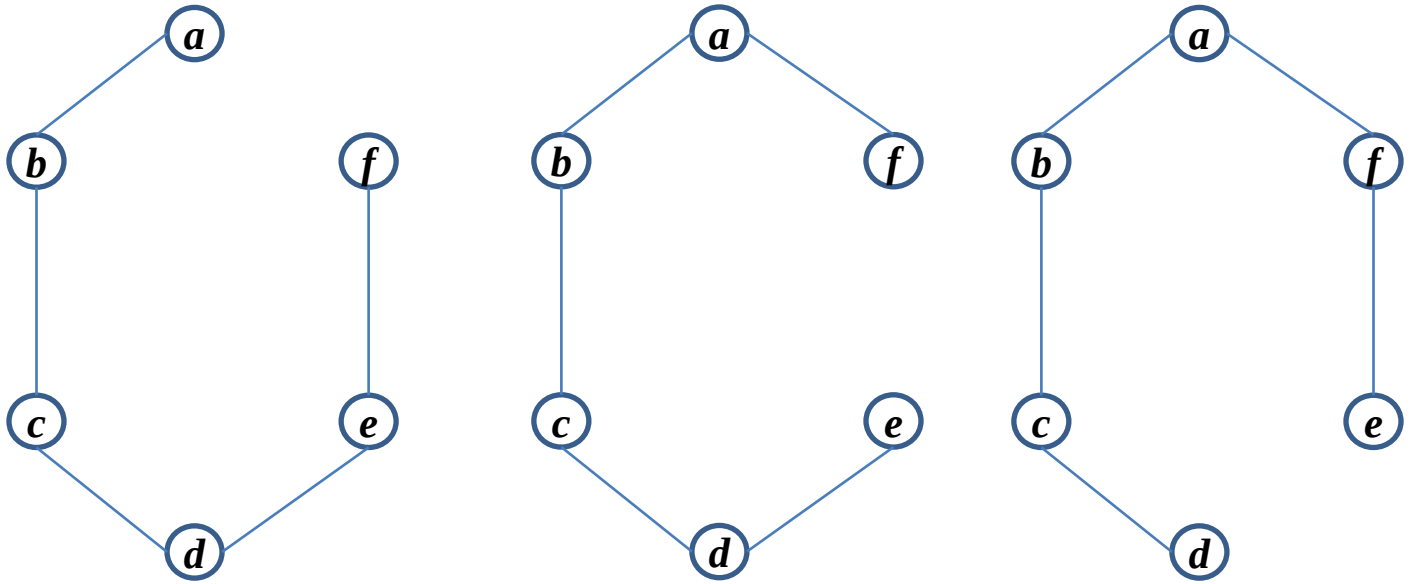
Source: http://www.uop.edu.pk/ocontents/Spanning%20Trees.pdf

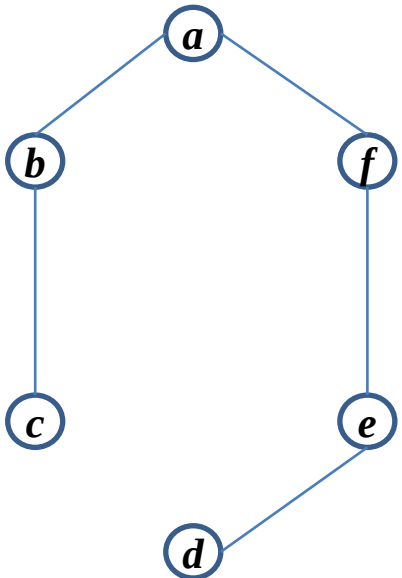
Q. Find all the spanning trees for a cycle C_6

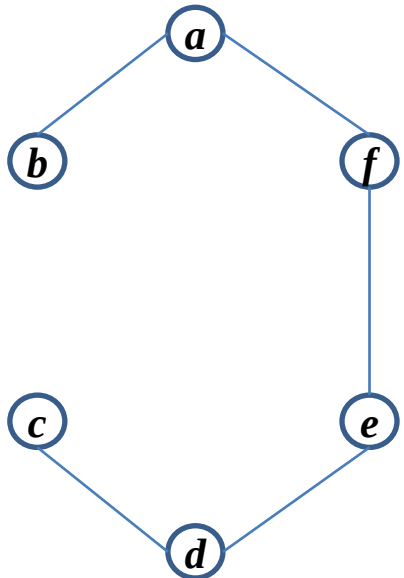
V={a,b,c,d,e,f}
|V| = 6
|E| = 6

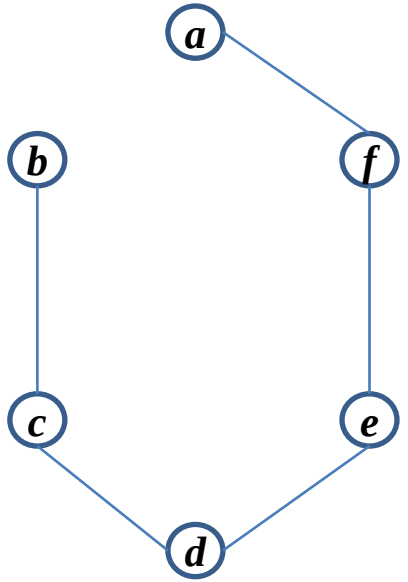


→ The diagram of all possible spanning trees from a graph C_6 are:









Minimum Spanning Tree(MST)

→ A wide variety of problems can be solved by finding a spanning tree in a weighted graph such that the sum of the weights of edges in the tree is a minimum.

→ A ***minimum spanning tree*** in a connected weighted graph is a spanning tree that has the ***smallest possible sum of weights*** of its edges.

OR

→ In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph.

ALGORITHM 1 Prim's Algorithm.

procedure *Prim*(G: weighted connected undirected graph with n vertices)
T := a minimum-weight edge
for $i := 1$ **to** $n - 2$
begin
  e := an edge of minimum weight incident to a vertex in T and not forming a simple circuit in T if added to T
  $T := T$ with e added
end {T is a minimum spanning tree of G}

ALGORITHM 2 Kruskal's Algorithm.

procedure *Kruskal*(G: weighted connected undirected graph with n vertices)
T := empty graph
for $i := 1$ **to** $n - 1$
begin
  e := any edge in G with smallest weight that does not form a simple circuit when added to T
  $T := T$ with e added
end {T is a minimum spanning tree of G}

Prim's Algorithm

```
Algorithm PrimMST(G)
{
 T = Φ; //T is a set of edges of MST
 S={s}; //s is randomly chosen vertex and S is set of
          vertices
 while(S!=V)
 {
    e=(u,v) an edge of minimum weight incident to
       vertices in S and not forming simple circuit in T
       if added to T i.e. u ∈ S and v ∈ V-S
    T=T ∪{(u,v)};
    S=S ∪ {v};
 }
}
```

Analysis:

In the above algorithm while loop executes $O(V)$. The edge of minimum weight incident on vertex can be found in $O(E)$, so the total time is $O(EV)$. We can improve the performance of the above algorithm by choosing better data structure as a priority Queue and normally it will be seen that the running time of Prim's Algorithm is $O(E \log V)$.

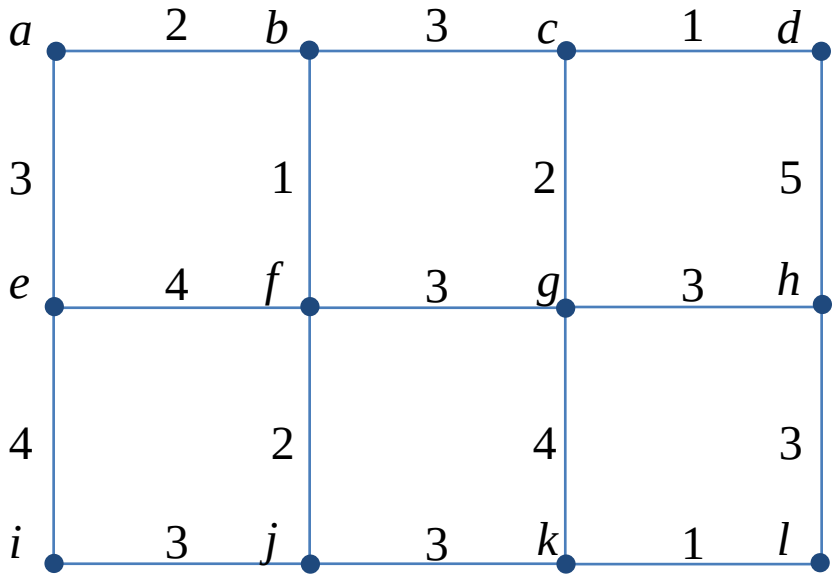
Kruskal's Algorithm

```
Algorithm KruskalMST(G)
{
T = Φ; //T is a set of edges of MST
E = set of edges sorted in non-decreasing order of weight
 while(|T|<n-1 and E!= Φ)
 {
        select (u,v) from E in order
        remove(u,v) from E
        if ((u,v) does not create a cycle in T)
                T = T U{(u,v)}
 }
}
```

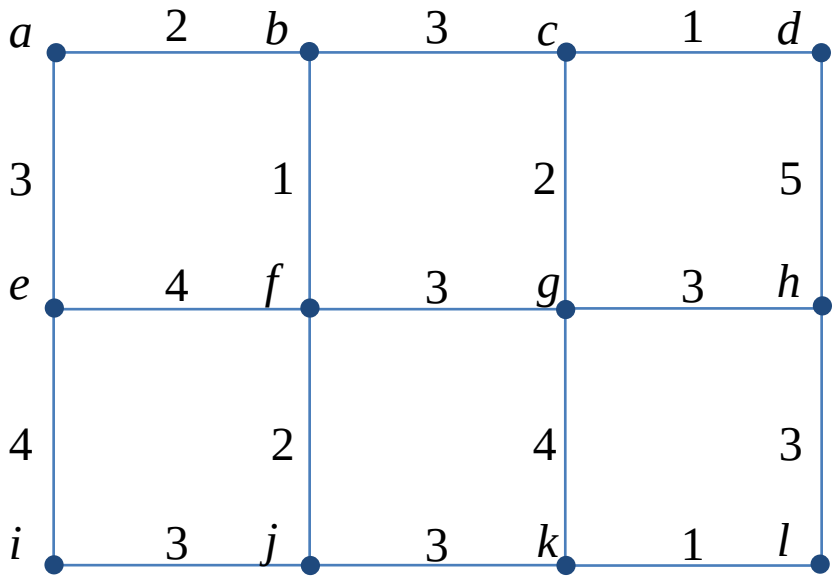
Analysis:

In the above algorithm, the initialization of the edge set requires sorting all edges, which takes $O(E \log E)$. The while loop then examines each edge in order, so it executes $O(E)$ times. For each edge, we must check whether adding it creates a cycle in the current tree set. If this cycle check is done naïvely, it can be costly, but the dominant term remains the sorting step. Therefore, the total running time of Kruskal's Algorithm is $O(E \log E)$, which is asymptotically equivalent to $O(E \log V)$.

Q_1. ***Draw the minimum spanning tree for the given graph***:



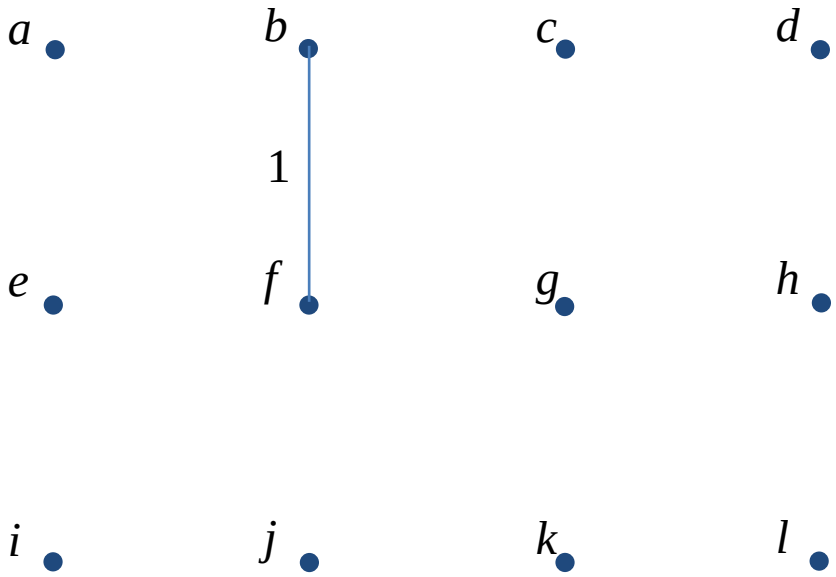
Solution using Prim's Algorithm:



Finding the weights of each edge:

Edge	***Weight***
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

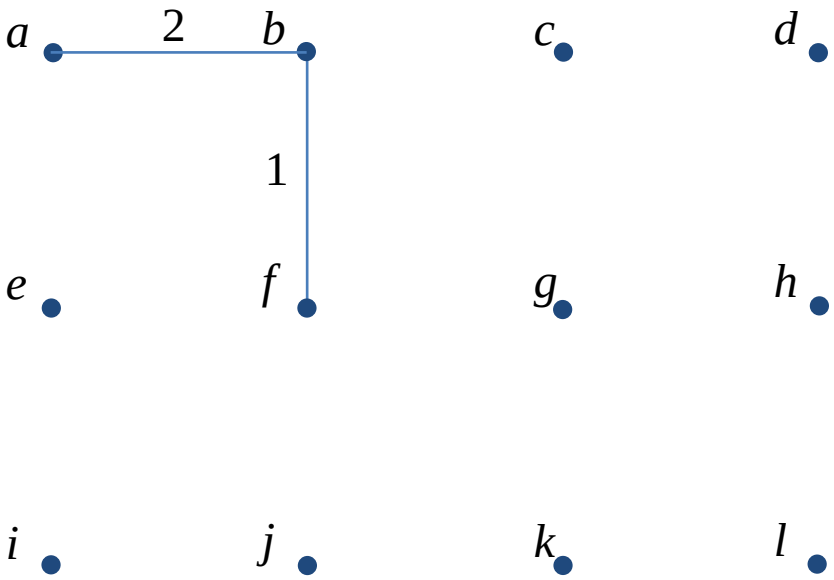
Select the edge with minimum weight.
Lets choose (b,f). Add the edge (b,f)



Next candidates edges:
(a,b): 2
(b,c): 3
(e,f): 4
(f,g): 3

Edge	***Weight***
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

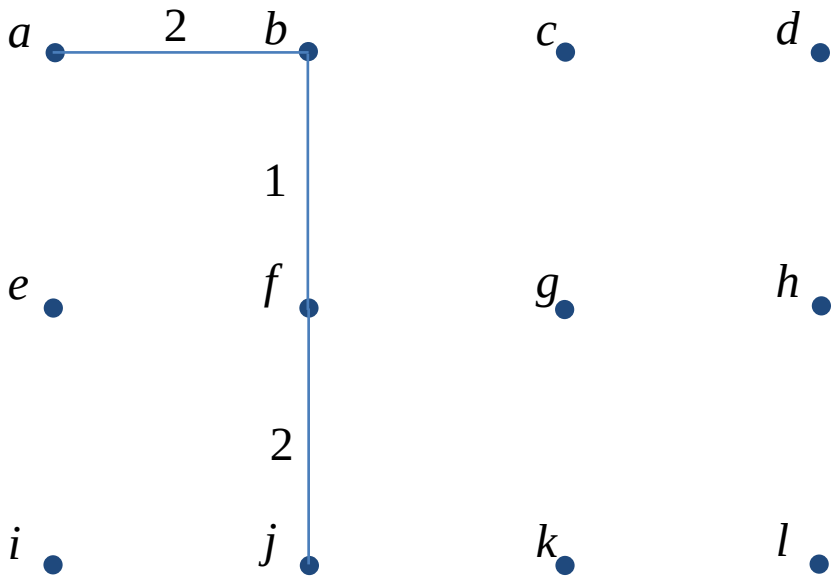
Add the edge (a,b)



Edge	Weight
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

Next candidates edges:
(a,e): 3
(e,f): 4
(b,c): 3
(f,g): 3
(f,j): 2

Add the edge (f,j)

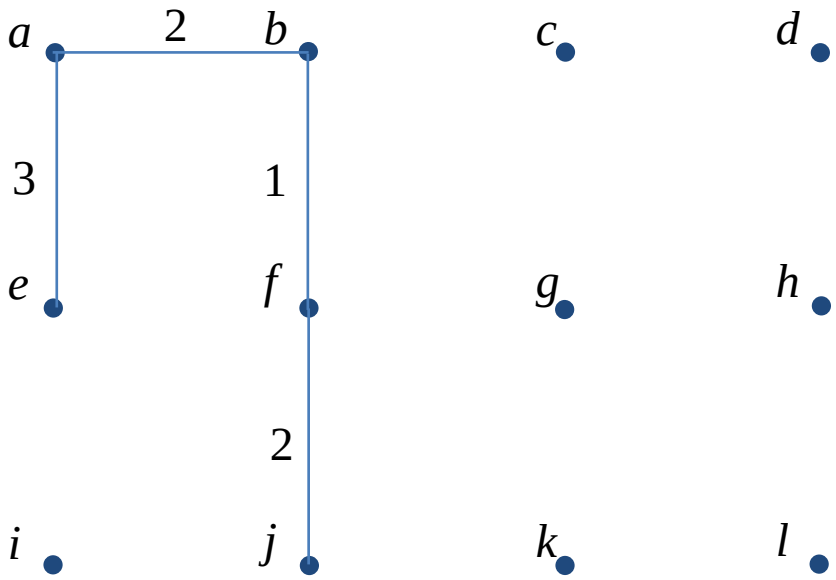


Next candidates edges:

(a,e): 2 *(b,c): 3*
(e,f): 4 *(f,g): 3*
(i,j): 3 *(j,k): 3*

Edge	***Weight***
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

Add the edge (a,e)



Edge	*Weight*
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

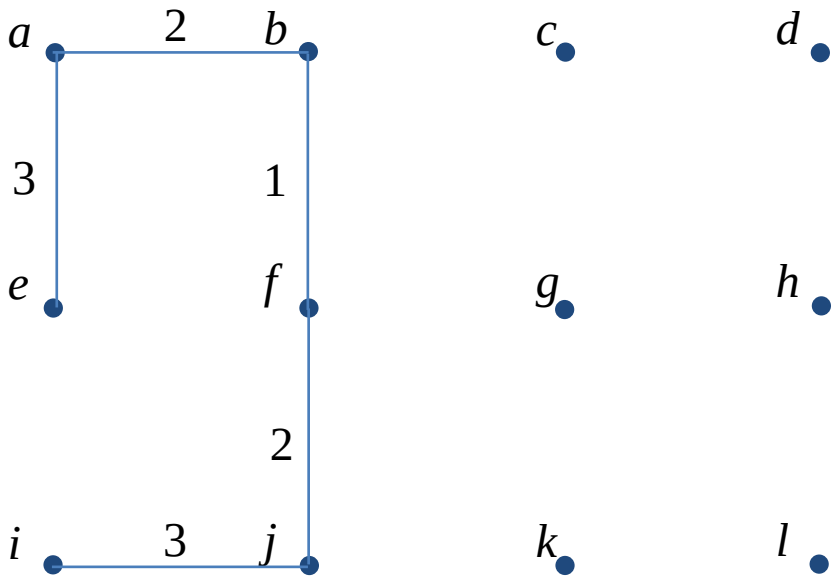
Next candidates edges:

(e,i): 4 *(f,g): 3*

(i,j): 3 *(j,k): 3*

(b,c): 3

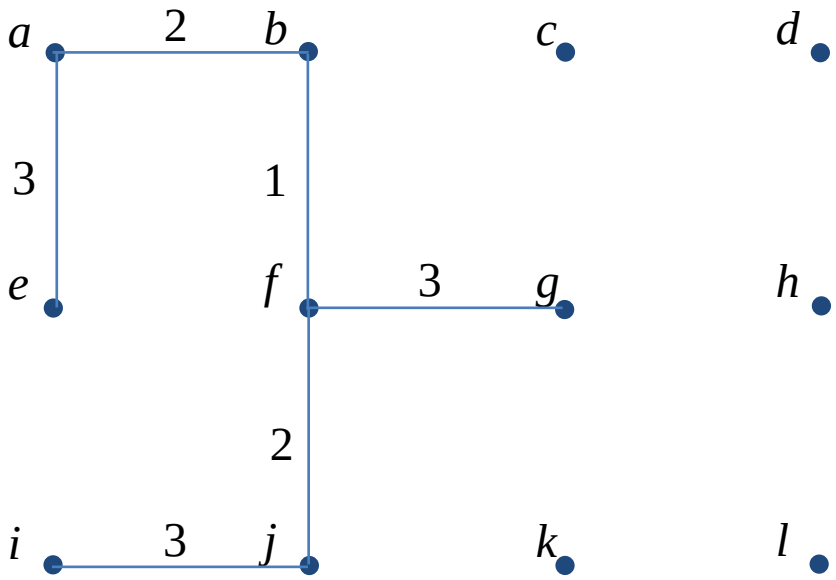
Add the edge (i,j)



Edge	*Weight*
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

Next candidates edges:
(f,g): 3
(b,c): 3
(j,k): 3

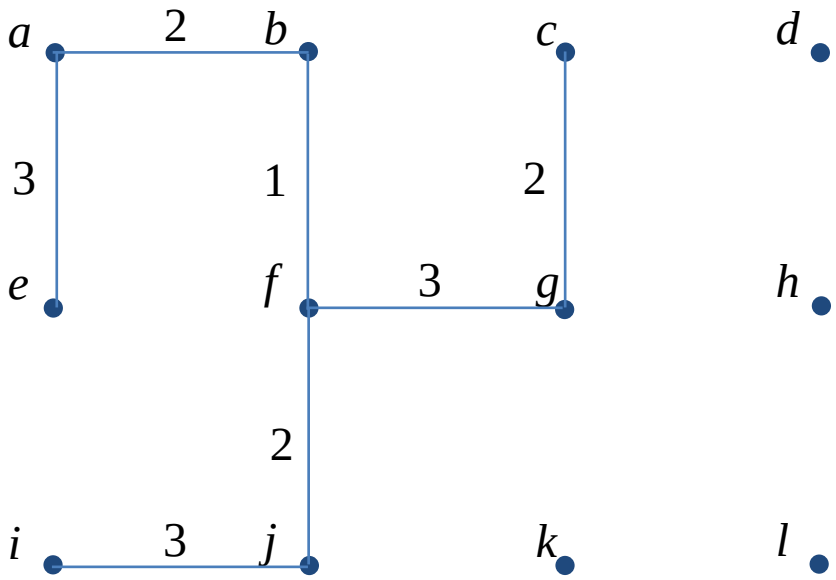
Add the edge (f,g)



Edge	*Weight*
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

Next candidates edges:
(b,c): 3 *(g,k): 4*
(c,g): 2
(j,k): 3

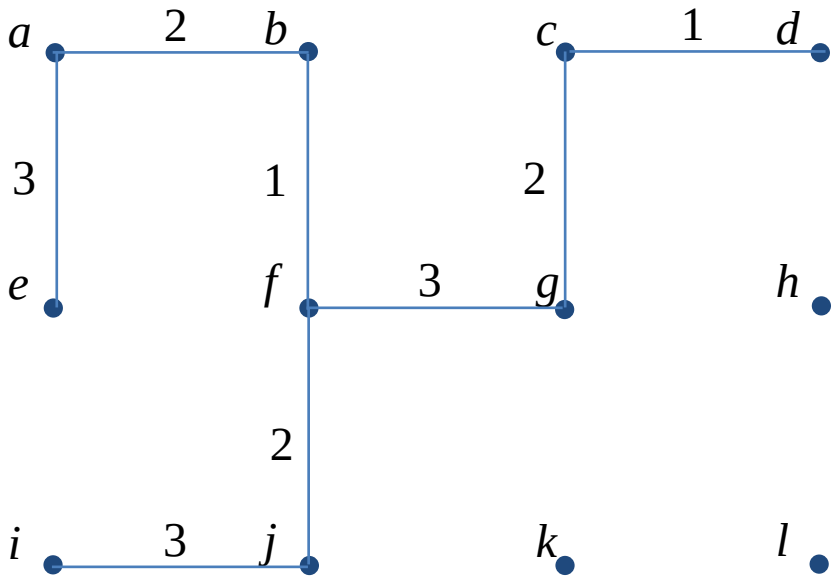
Add the edge (c,g)



Next candidates edges:
(g,k): 4 *(g,h): 3*
(j,k): 3
(c,d): 1

Edge	***Weight***
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

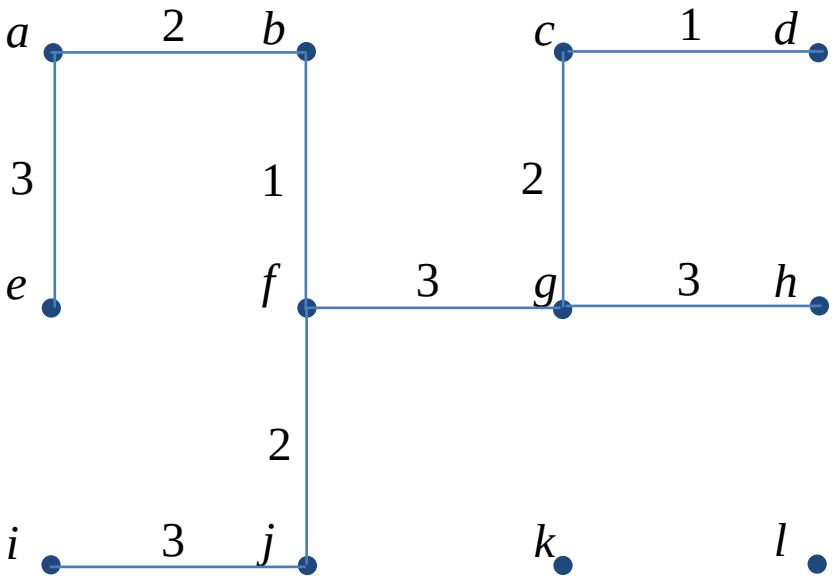
Add the edge (c,d)



Next candidates edges:
(g,h): 3 *(d,h): 5*
(g,k): 4
(j,k): 3

Edge	***Weight***
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

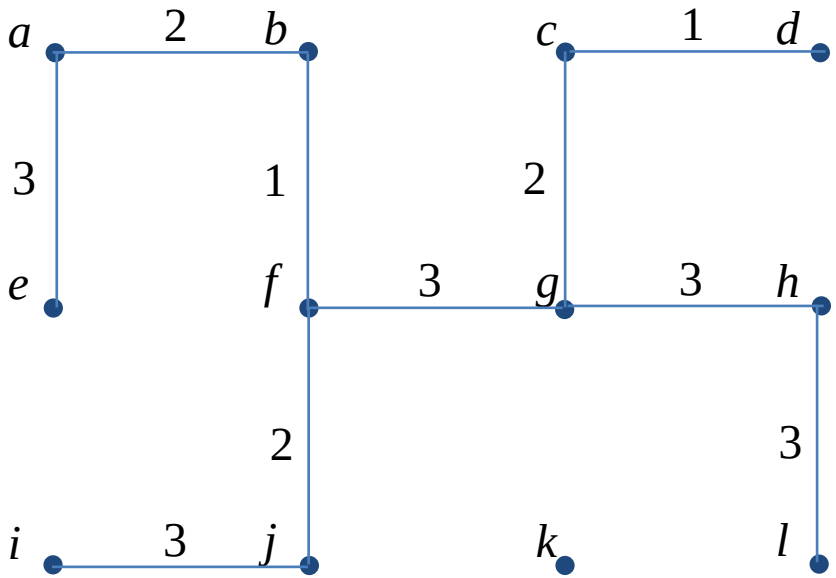
Add the edge (g,h)



Next candidates edges:
(h,l): 3
(j,k): 3
(g,k): 4

Edge	***Weight***
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

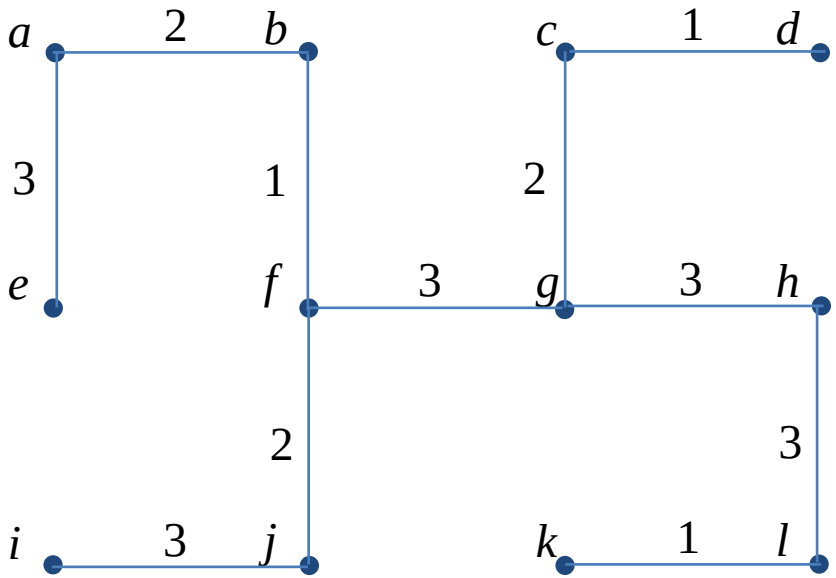
Add the edge (h,l)



Next candidates edges:
(k.l): 1
(j,k): 3

Edge	***Weight***
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

Add the edge (k,l)



Next candidates edges:
No possible edges.

Edge	***Weight***
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

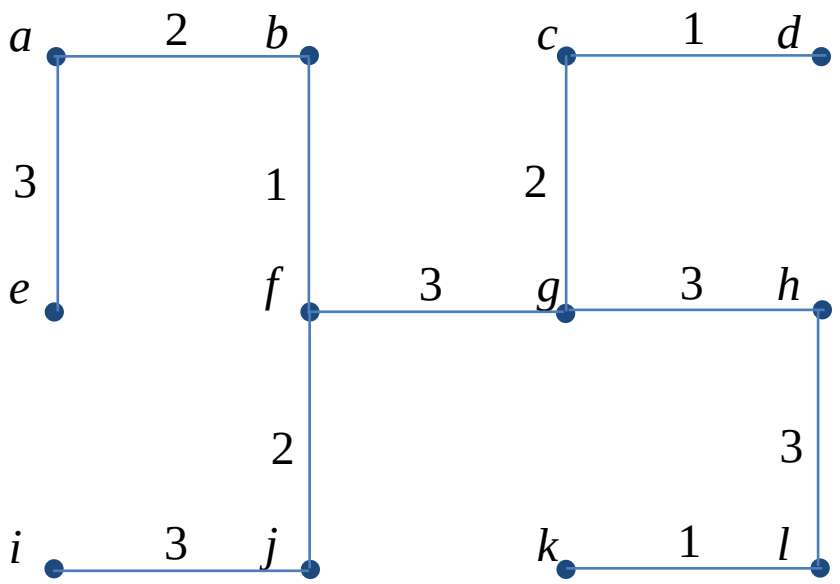
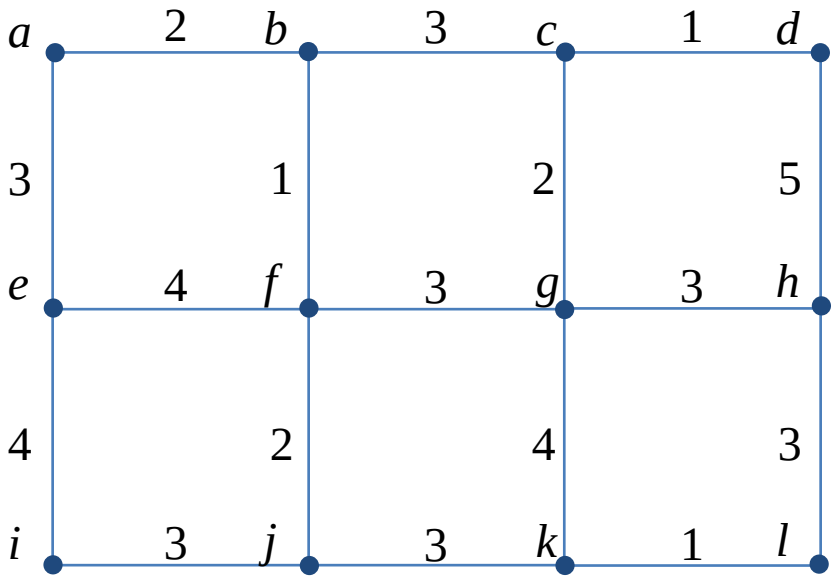


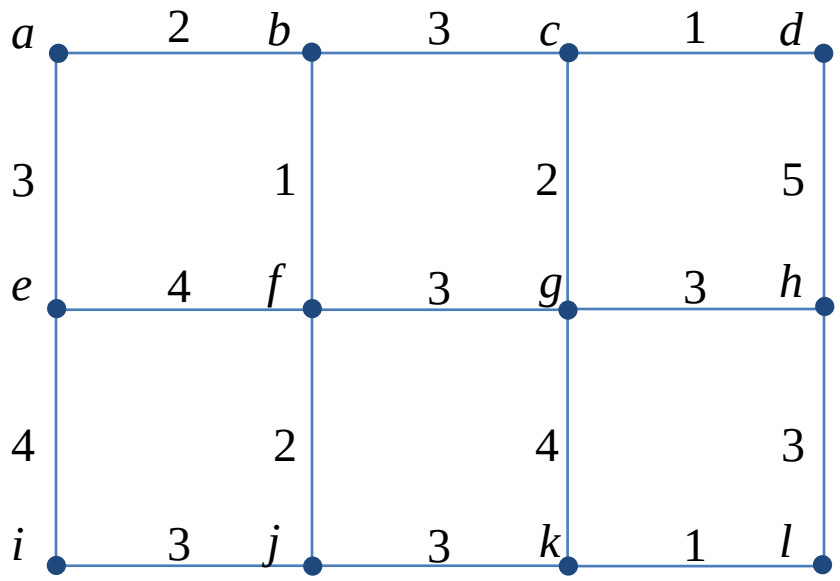
Figure: ***Minimum Spanning Tree***

Total weight of minimum spanning tree = **24**

Solution using Kruskal's Algorithm:

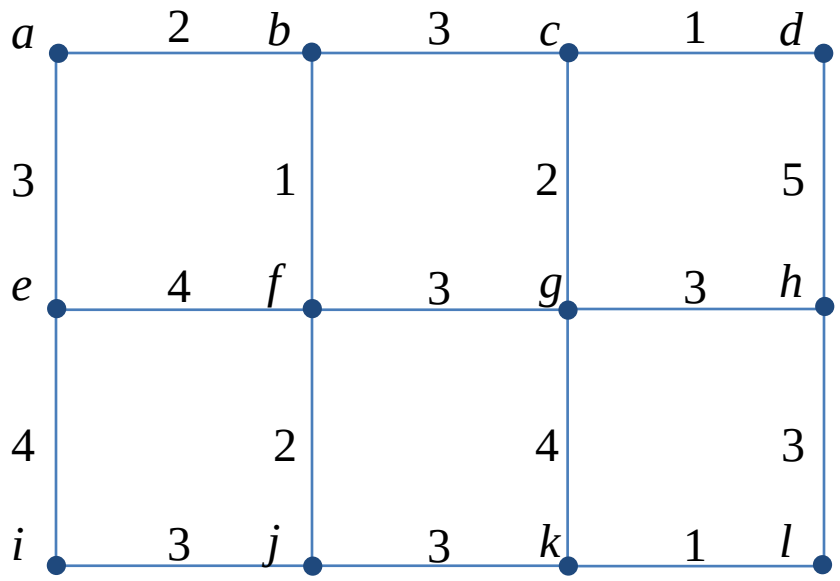


Finding the weights of each edge:



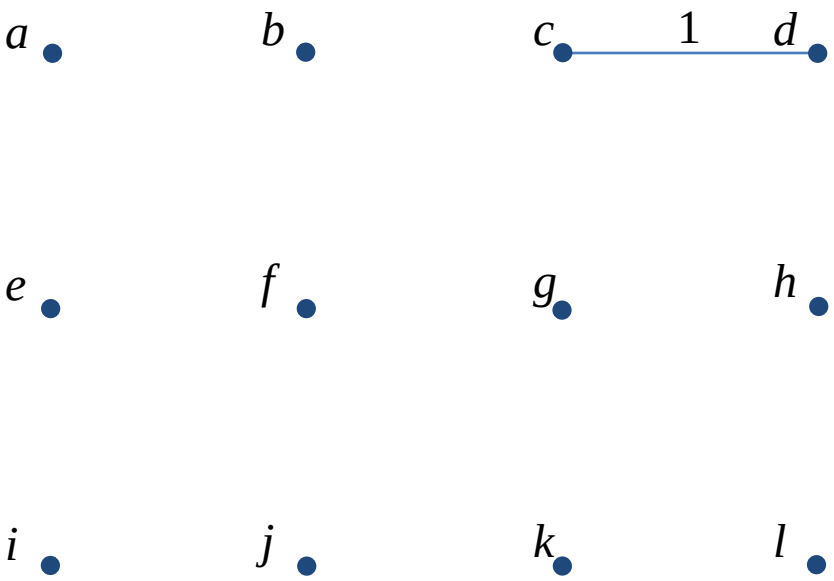
Edge	***Weight***
(a,b)	2
(b,c)	3
(c,d)	1
(e,f)	4
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(k,l)	1
(a,e)	3
(e,i)	4
(b,f)	1
(f,j)	2
(c,g)	2
(g,k)	4
(d,h)	5
(h,l)	3

Arranging the edges in ascending order of weights:



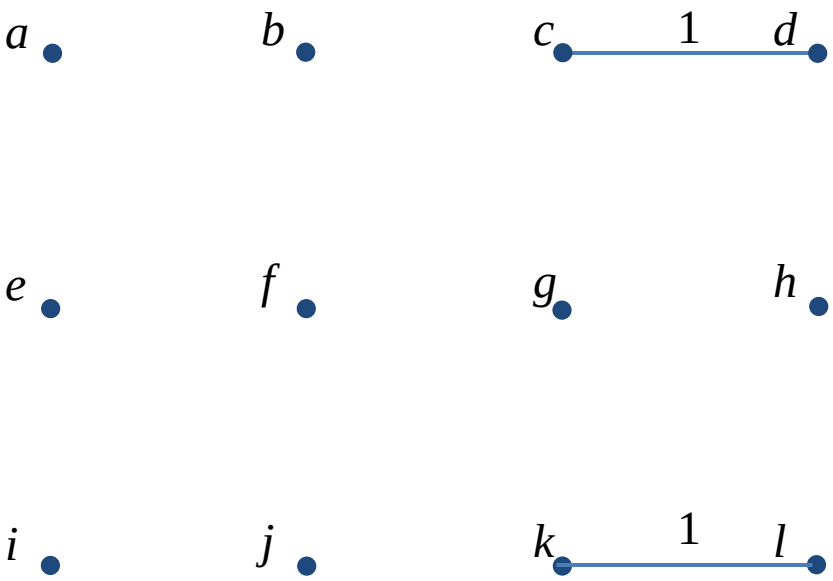
Edge	***Weight***
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (c,d):



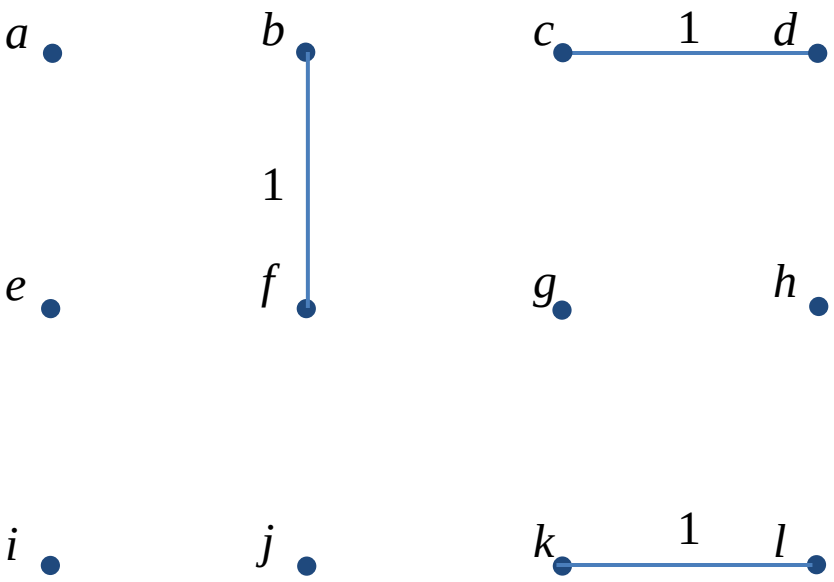
Edge	***Weight***
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (k,l):



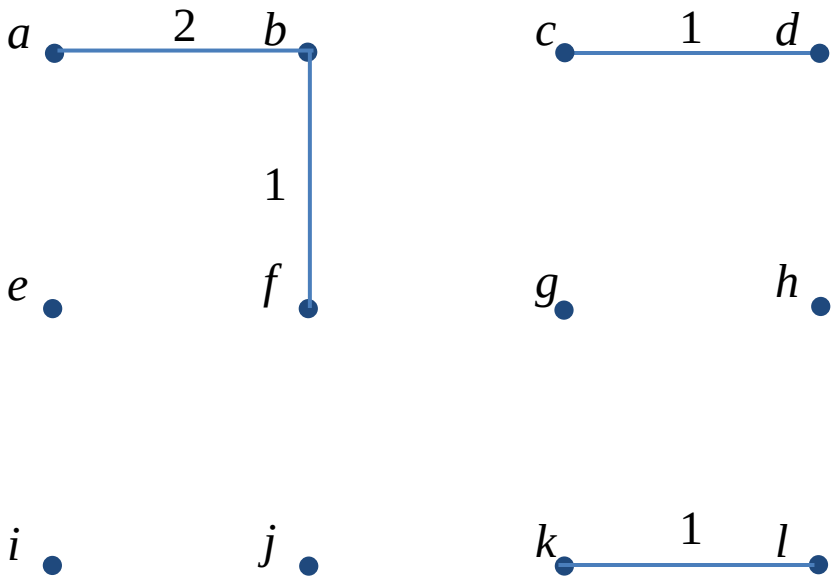
Edge	***Weight***
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (b,f):



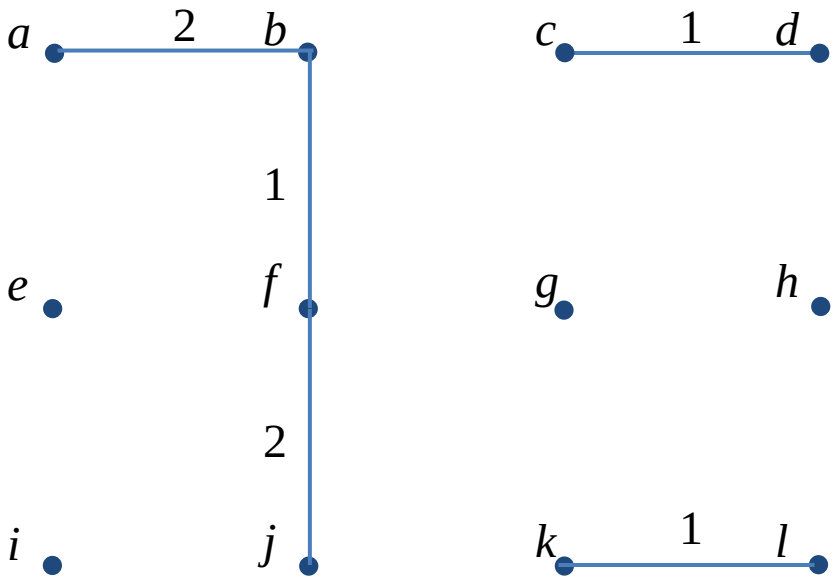
Edge	***Weight***
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (a,b):



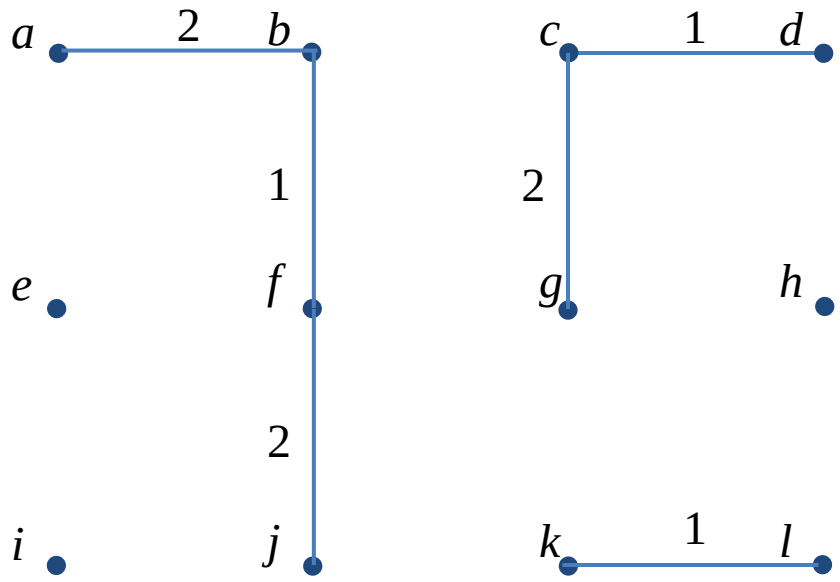
Edge	*Weight*
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (f,j):



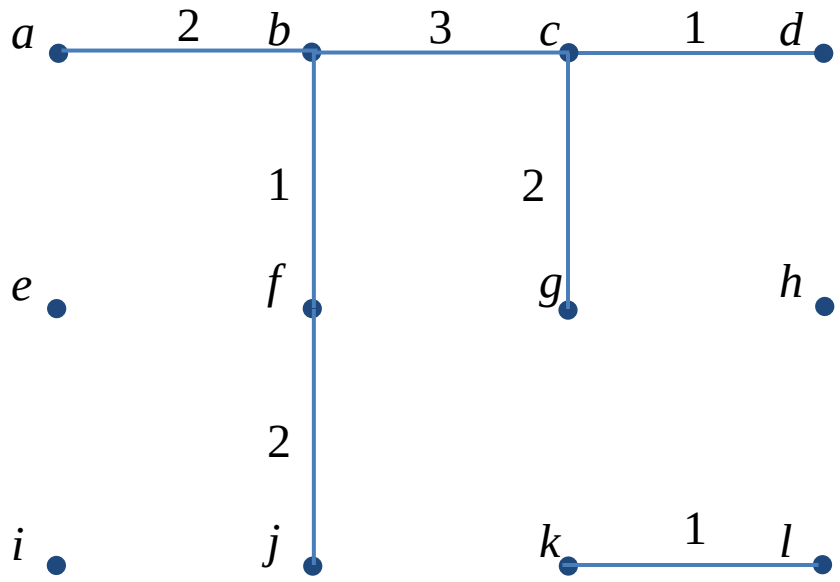
Edge	***Weight***
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (c,g):



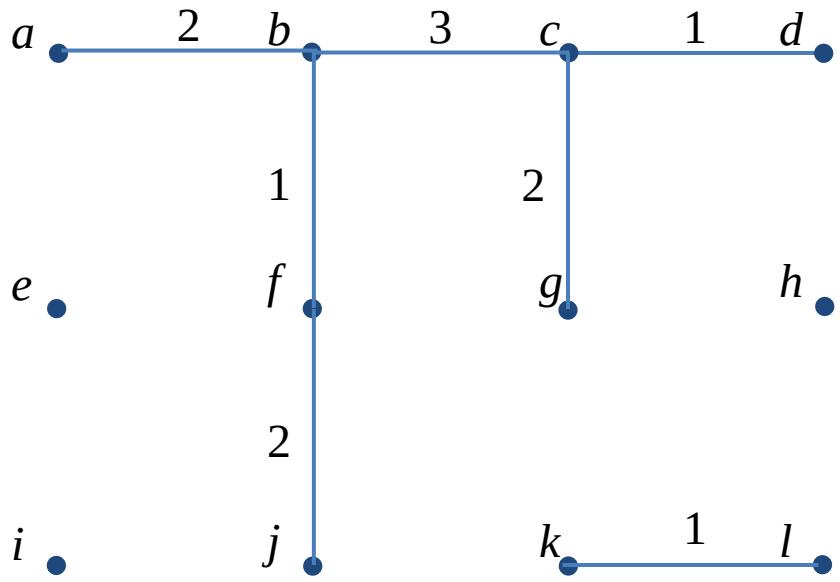
Edge	*Weight*
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (b,c):



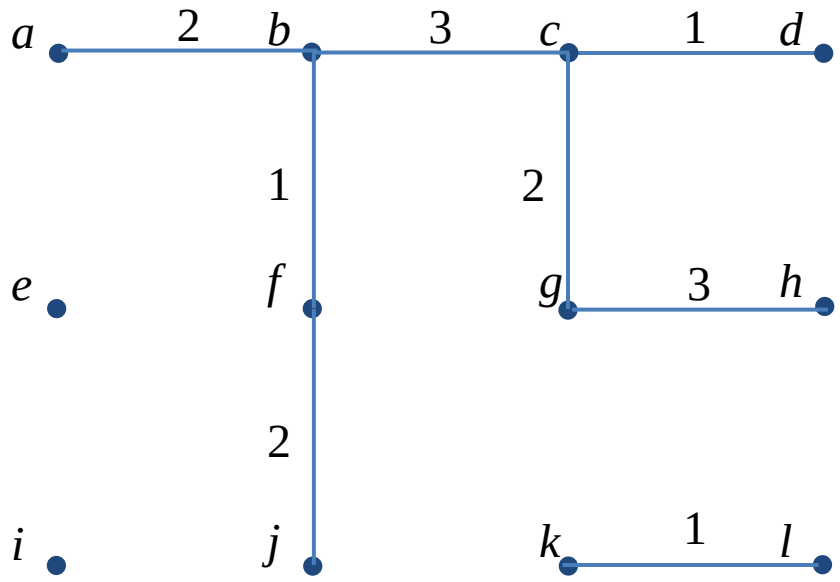
Edge	***Weight***
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Ignoring edge (f,g) because it forms a cycle:



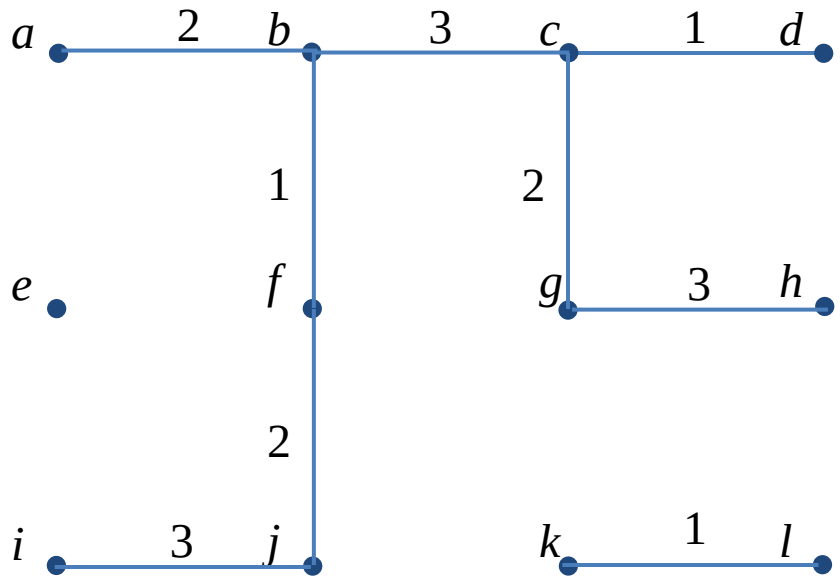
Edge	*Weight*
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (g,h):



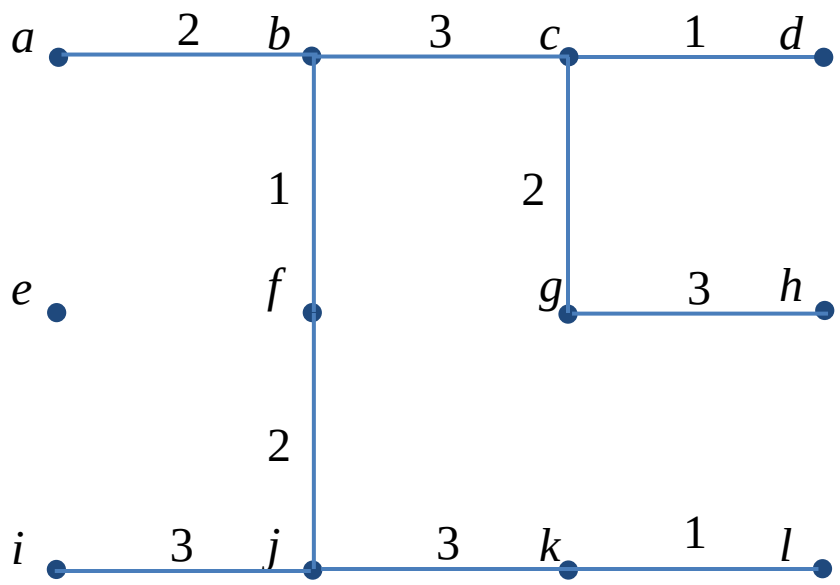
Edge	*Weight*
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (i,j):



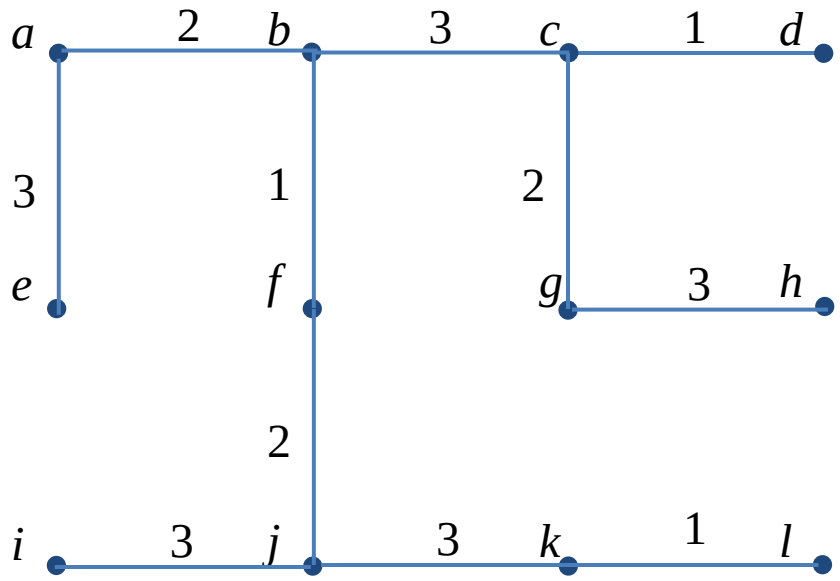
Edge	*Weight*
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (j,k):



Edge	*Weight*
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Adding edge (a,e):



Edge	*Weight*
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g)	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l)	3
(e,f)	4
(e,i)	4
(g,k)	4
(d,h)	5

Ignoring edges (h,l), (e,f), (e,i), (g,k) and (d,h) because they form a cycle.:

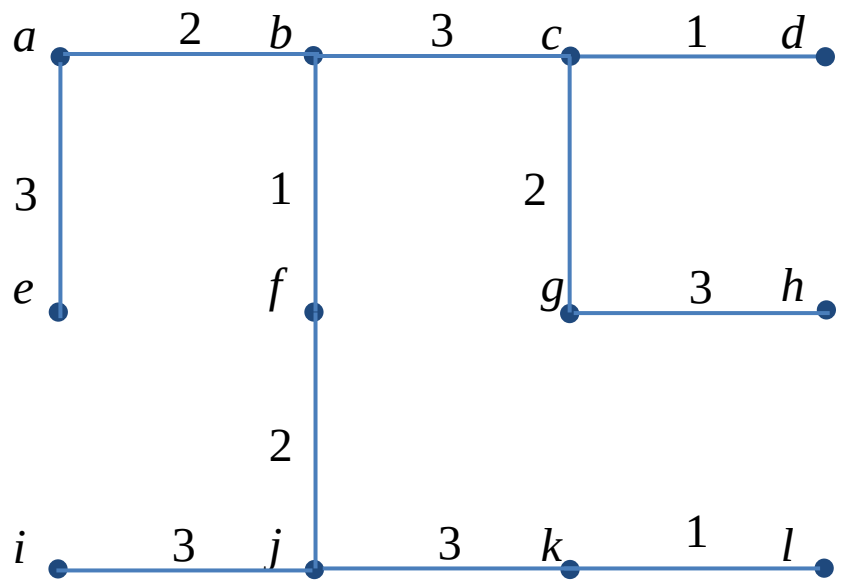
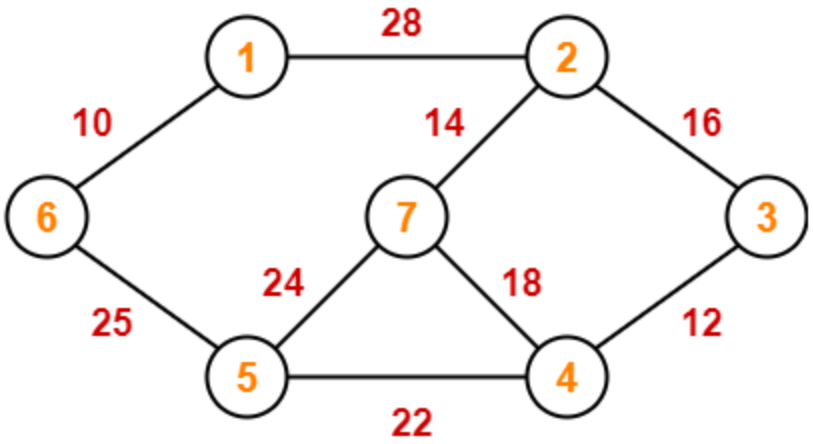


Figure: ***Minimum spanning tree***

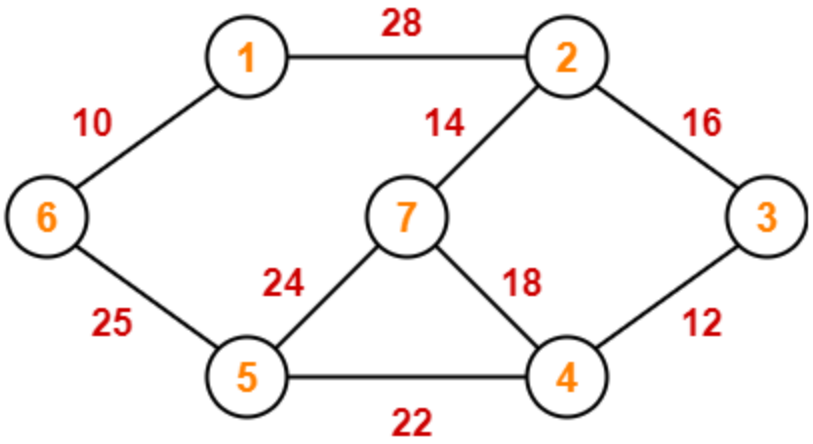
Edge	***Weight***
(c,d)	1
(k,l)	1
(b,f)	1
(a,b)	2
(f,j)	2
(c,g)	2
(b,c)	3
(f,g) ×	3
(g,h)	3
(i,j)	3
(j,k)	3
(a,e)	3
(h,l) ×	3
(e,f) ×	4
(e,i) ×	4
(g,k) ×	4
(d,h) ×	5

Total weight of minimum spanning tree = **24**

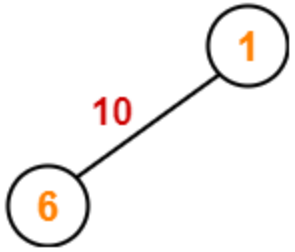
$Q_{2.}$ Find the minimum spanning tree in the following graph using Prim's Algorithm:



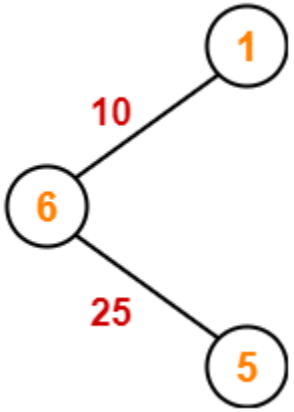
Solution:

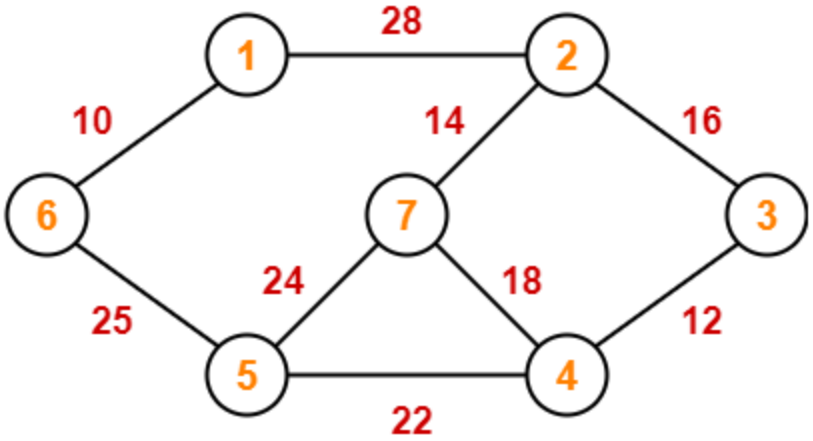


Step 1

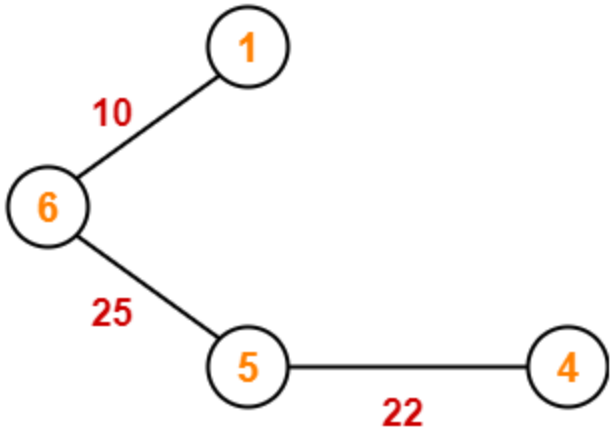


Step 2

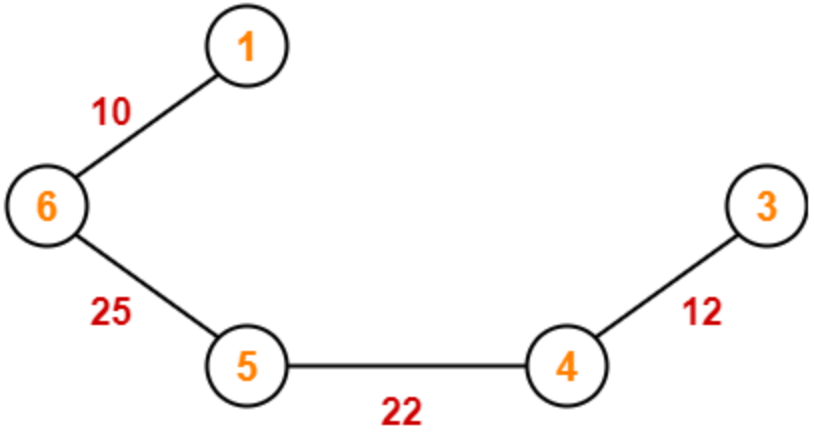


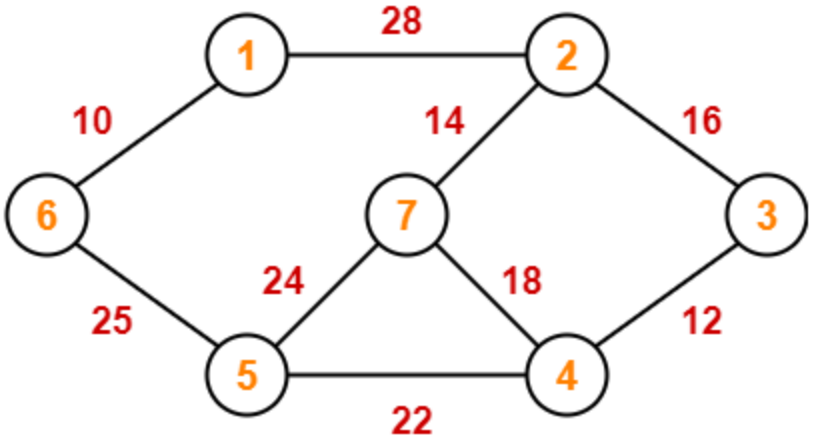


Step 3

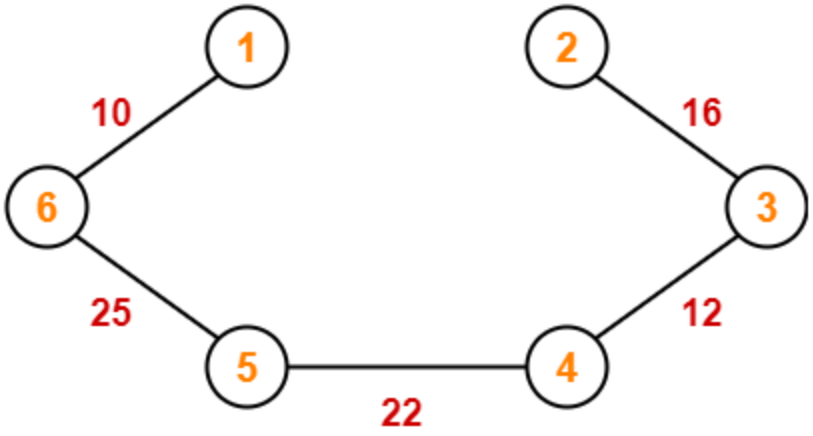


Step 4

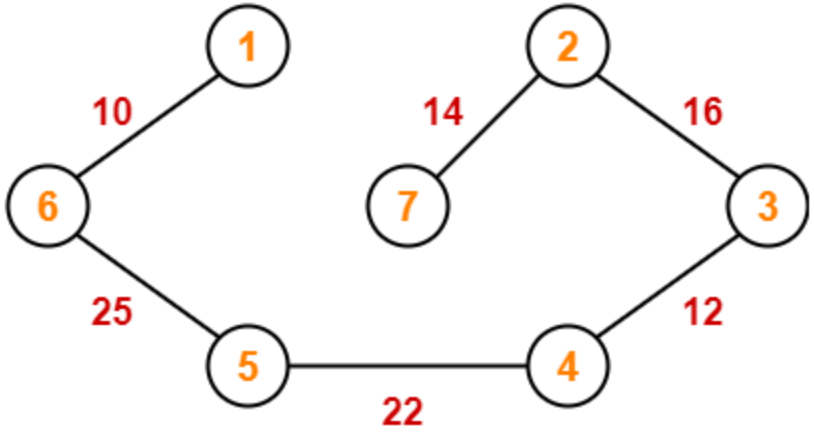




Step 5

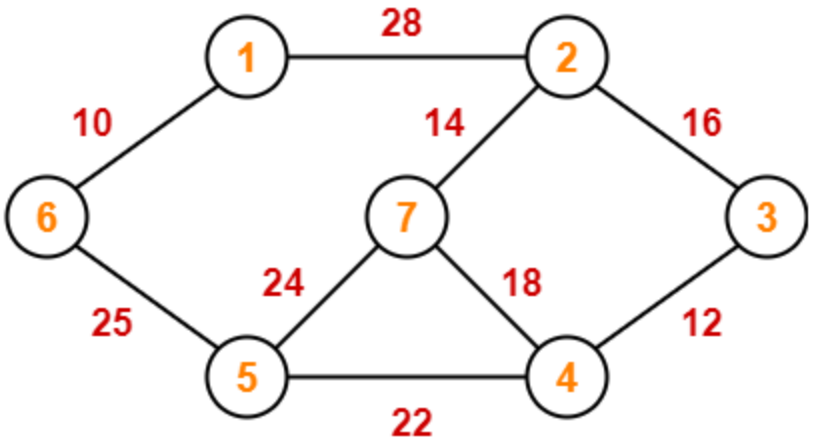


Step 6

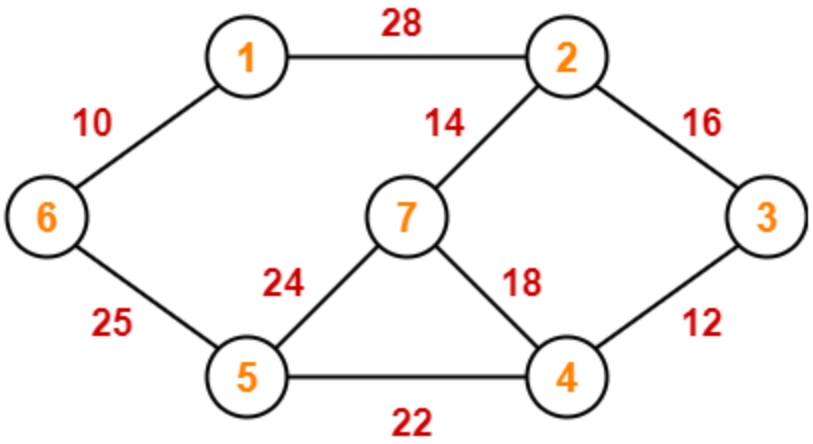


Weight of MST
= 10+25+22+12+16+14
= 99

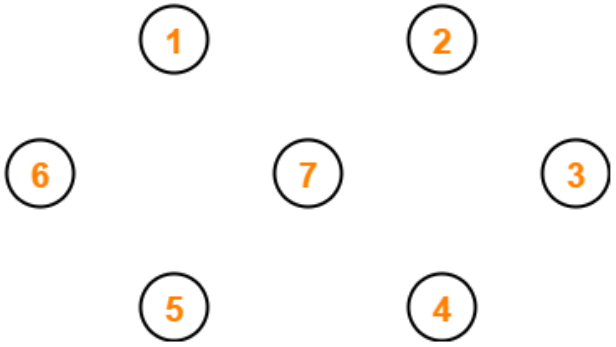
$Q_{3.}$ Find the minimum spanning tree in the following graph using Kruskal's Algorithm:



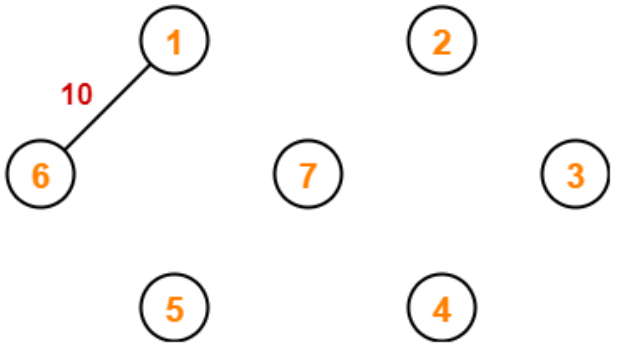
Solution:

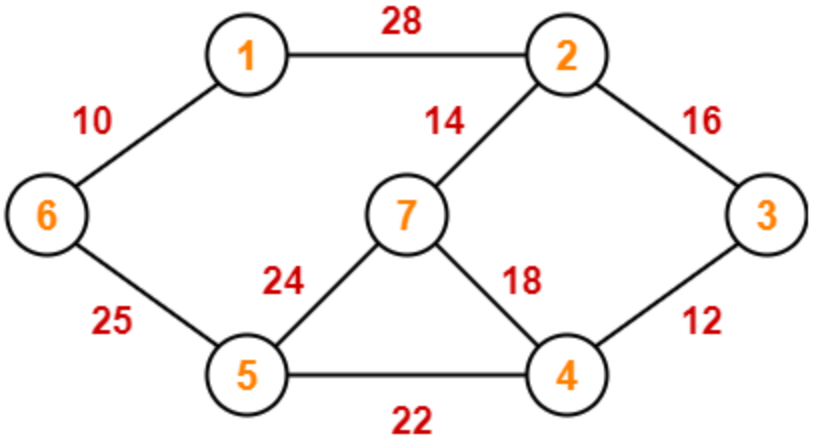


Step 1

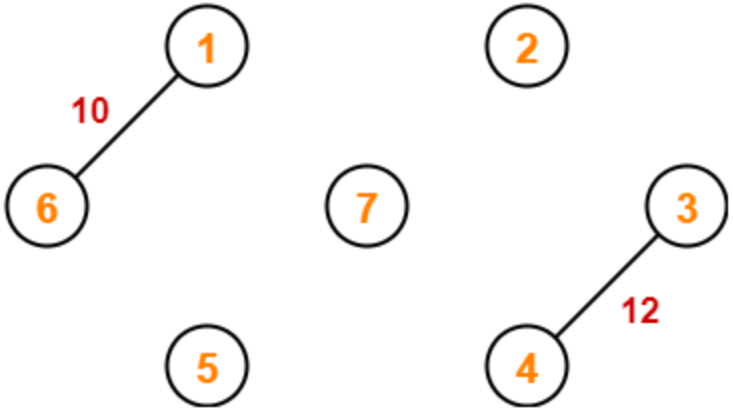


Step 2

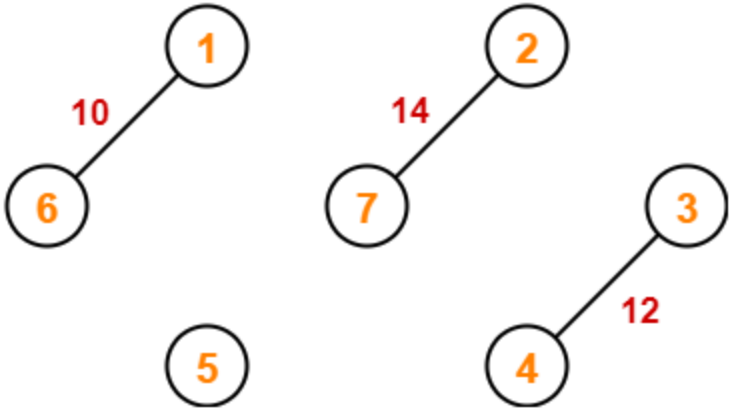


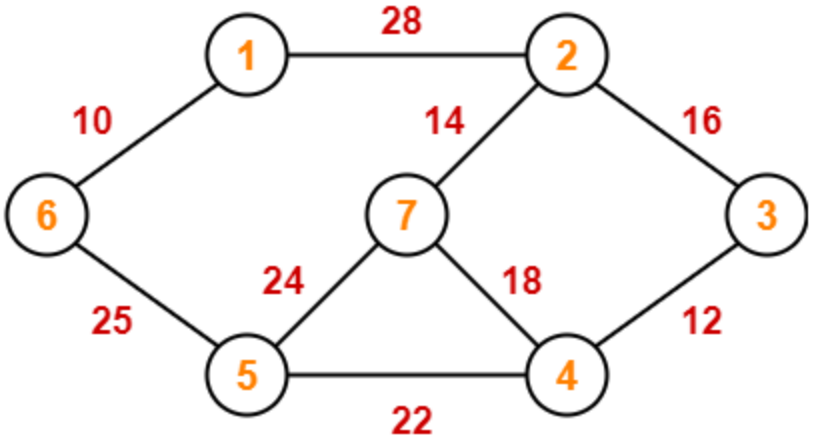


Step 3

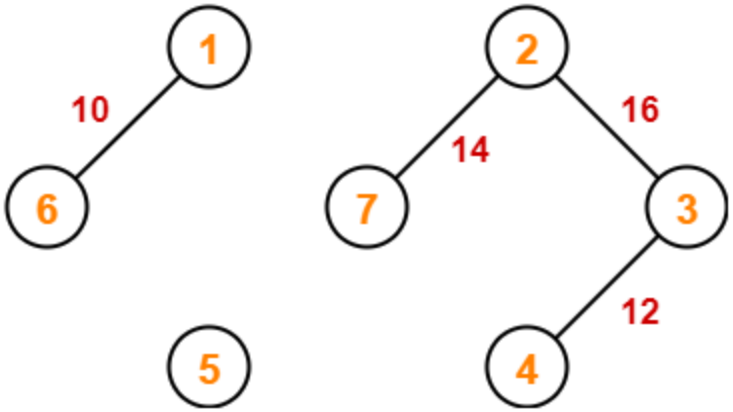


Step 4

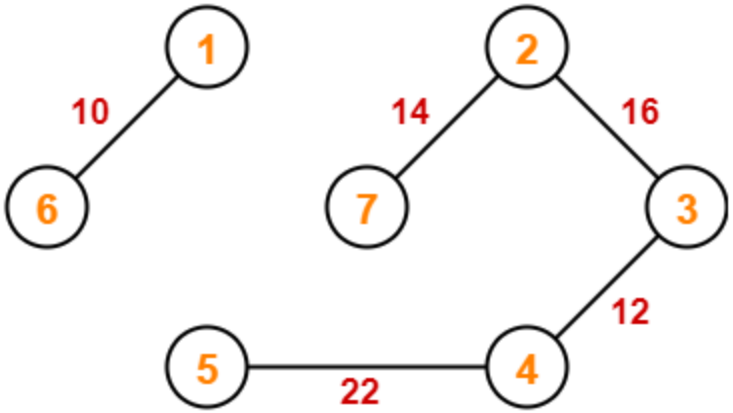




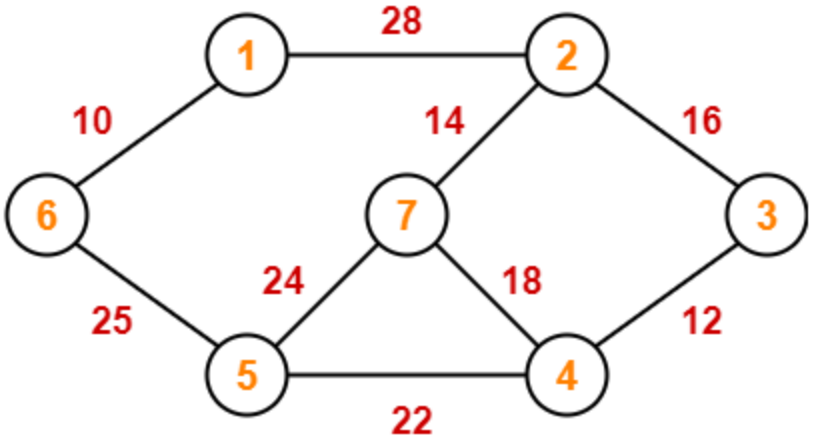
Step 5



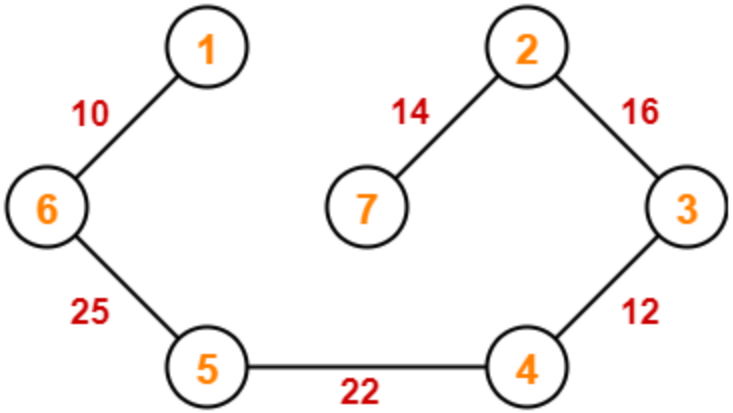
Step 6



Weight of MST
= 10+25+22+12+16+14
= 99

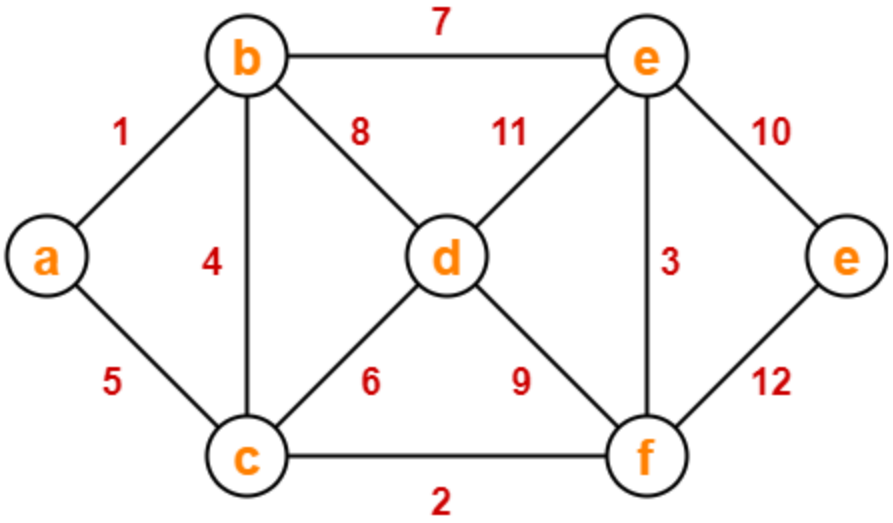


Step 7

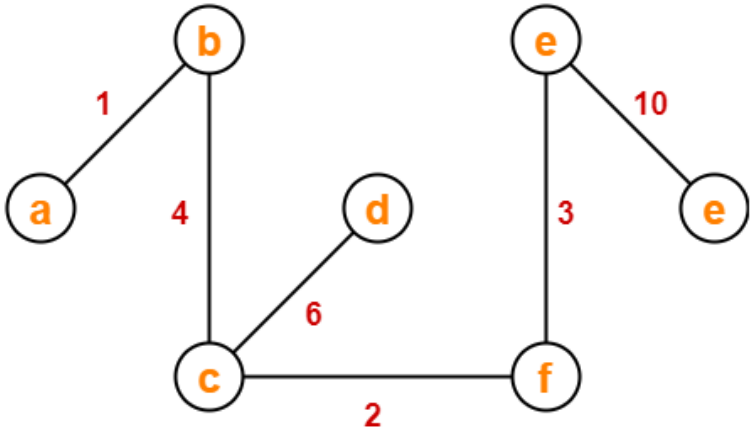


Weight of MST
= 10+25+22+12+16+14
= 99

Q_3. Find the minimum spanning tree in the following graph using Prim's Algorithm:



Answer:



Weight of MST = 26

Thank You!

Any Queries?